Algoritmos Avançados

Aulas 27 & 28: Caminho Mínimo em Grafos – Do BFS ao Algoritmo de Dijkstra

Prof. Me. Douglas Samuel Gonçalves

Fatec Bebedouro – Jorge Caram Sabbag
Tecnologia em Big Data no Agronegócio

2º Semestre / 2026

Sumário da Aula

1. Motivação & O Problema do Caminho Mínimo
2. Caminho Mínimo em Grafos Não-Ponderados
3. A Falha do BFS & O Princípio do Relaxamento
4. O Algoritmo de Dijkstra
5. Implementação Orientada a Objetos em Python
6. Estudos de Caso Práticos no Agronegócio
7. Exercícios Resolvidos & Desafios
8. Próximos Passos & Encerramento

O Problema Central: Encontrar a Rota Ótima

Na era do Big Data e da Agricultura de Precisão, redes de transporte, telecomunicações e sensores exigem movimentação com custo mínimo.

Definição Intuitiva

Dado um grafo com vértices interligados por rotas, encontrar a sequência de trechos que conecta uma **Origem** a um **Destino** com:

- Menor tempo de trânsito (minutos);
- Menor consumo de combustível / diesel (litros);
- Menor frete total rodoviário (R$);
- Menor número de saltos de rádio em redes IoT.

Impacto Direto no Agronegócio

- **Cana-de-Açúcar:** O tempo entre o corte e a moagem na usina não pode exceder 24h para evitar perda de sacarose (ATR).
- **Logística de Grãos:** Redução de centavos por tonelada transportada em carretas bitrem representa milhões de reais de lucro na safra.

Formulação Matemática do Caminho Mínimo

Definition (Peso de um Caminho)

Seja $G = (V, E)$ um grafo e $w : E \to \mathbb{R}$ uma função que atribui peso a cada aresta. O peso de um caminho $p = \langle v_0, v_1, v_2, \ldots, v_k \rangle$ é a soma dos pesos de suas arestas:

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

Distância de Caminho Mínimo ($\delta(u, v)$)

A distância $\delta(u, v)$ de u até v é o peso do caminho de menor custo entre eles:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \xrightarrow{p} v\} & \text{se existe caminho de } u \text{ até } v \\ \infty & \text{caso contrário} \end{cases}$$

Propriedade Fundamental: Subestrutura Ótima

Os problemas de caminho mínimo exibem uma propriedade matemática determinante que viabiliza tanto algoritmos gulosos quanto programação dinâmica:

Lema da Subestrutura Ótima (Cormen et al.)

Seja $p = \langle v_0, v_1, \ldots, v_k \rangle$ um caminho mínimo entre v_0 e v_k. Para quaisquer índices intermediários i e j tais que $0 \leq i \leq j \leq k$:

O subcaminho $p_{ij} = \langle v_i, v_{i+1}, \ldots, v_j \rangle$ é também um caminho mínimo entre v_i e v_j!



Prova por Contradição (Cortar e Colar):
Se existisse um caminho mais curto entre v_i e v_j, poderíamos substituir o trecho intermediário e obter um caminho total menor de v_0 a v_k, o que contradiz a minimalidade original de p.

Variantes do Problema em Engenharia de Dados

1. Origem Única (SSSP)

Single-Source Shortest Path:

- Dado um nó inicial s, calcula a menor distância de s para **todos os outros vértices** do grafo.
- **Algoritmos:** BFS (não-ponderado), Dijkstra ($w \geq 0$), Bellman-Ford ($w \in \mathbb{R}$).
- Foco principal desta aula!

2. Todos os Pares (APSP)

All-Pairs Shortest Paths:

- Calcula a menor distância entre **qualquer par** de nós $(u, v) \in V \times V$.
- Produz uma matriz $|V| \times |V|$ de distâncias.
- **Algoritmos:** Floyd-Warshall ($\mathcal{O}(V^3)$), Johnson ($\mathcal{O}(V^2 \log V + VE)$).

Curiosidade Prática

Não existe nenhum algoritmo assintoticamente mais rápido para encontrar o caminho para um único par (s, t) do que resolver o problema para todos a partir de s (SSSP)!

O Caso Especial Não-Ponderado ($w \equiv 1$)

Quando todas as conexões possuem o mesmo custo unitário (ou custo homogêneo), o peso de um caminho é simplesmente o **número de arestas (saltos)**:

$$w(p) = |p| = \text{número de arestas}$$

Teorema da Busca em Largura (BFS)

Em qualquer grafo não-ponderado G, o BFS inicializado em s:

- Descobre os vértices em ordem crescente de distância;
- Calcula rigorosamente $d[v] = \delta(s, v)$ para todo $v \in V$;
- Opera em tempo ótimo linear: $\mathcal{O}(|\mathbf{V}| + |\mathbf{E}|)$.

Estrutura Auxiliar

A Fila FIFO (First-In, First-Out) garante que todos os nós a distância k sejam processados antes de qualquer nó a distância $k + 1$.

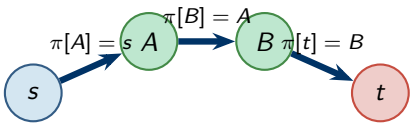
Reconstrução da Rota via Predecessores (π)

O vetor de distâncias $d[v]$ informa apenas *"qual é o custo"*, mas não diz *"por onde passar"*. Para isso, mantemos o vetor de pais $\pi[v]$:

Mecanismo de Backtracking

Para recuperar o caminho ótimo da Origem s ao Destino t:

1. Inicie em t;
2. Retroceda sucessivamente: $t \leftarrow \pi[t]$;
3. Pare quando alcançar s ($\pi[s] = \text{None}$);
4. Inverta a lista resultante.



Caminho Mínimo Reconstruído:
$\langle s, A, B, t \rangle$ (3 saltos)

Métricas Topológicas Globais: Excentricidade

A partir de caminhos mínimos, definimos medidas quantitativas fundamentais da estrutura de qualquer rede:

Definition (Excentricidade de um Vértice)

A **Excentricidade** $\epsilon(v)$ de um vértice v é a maior distância de caminho mínimo de v até qualquer outro vértice da rede:

$$\epsilon(v) = \max_{u \in V} \delta(v, u)$$

Interpretação Prática

Mede o **"pior caso"** de alcance a partir daquele nó.

- Vértices na periferia/borda da rede possuem **alta** excentricidade;
- Vértices centrais/hubs possuem **baixa** excentricidade.

Raio, Diâmetro e Centro do Grafo

Raio do Grafo ($\text{rad}(G)$)

A menor das excentricidades entre todos os vértices:

$$\text{rad}(G) = \min_{v \in V} \epsilon(v)$$

Diâmetro do Grafo ($\text{diam}(G)$)

A maior das excentricidades (maior distância mínima entre dois nós quaisquer):

$$\text{diam}(G) = \max_{v \in V} \epsilon(v)$$

Centro do Grafo ($Z(G)$)

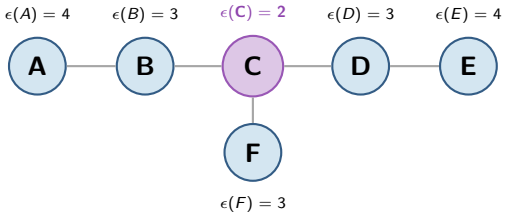
O conjunto de todos os vértices cuja excentricidade atinge o valor mínimo (o raio):

$$Z(G) = \{v \in V \mid \epsilon(v) = \text{rad}(G)\}$$

Propriedade: Em qualquer grafo conexo:

$$\text{rad}(G) \leq \text{diam}(G) \leq 2 \cdot \text{rad}(G)$$

Exemplo Visual: Cálculo das Métricas de Rede



Métricas Calculadas

- $\epsilon(A) = \delta(A, E) = 4$
- $\epsilon(C) = \max\{2, 2, 1\} = \mathbf{2}$
- Raio $\text{rad}(G) = \mathbf{2}$
- Diâmetro $\text{diam}(G) = \mathbf{4}$
- **Centro** $Z(G) = \{\mathbf{C}\}$

C é o ponto ideal para instalação de infraestrutura central!

Decisão de Engenharia: Onde Construir o Silo?

Problema da Localização de Facilidades (1-Center Problem)

Uma cooperativa com unidades em 10 cidades precisa construir um Armazém Geral de Defensivos e Peças Agrícolas. Onde instalá-lo?

Critério Minimax (Centro do Grafo)

Minimiza o pior tempo de atendimento a qualquer associado da cooperativa:

$$\min_{v \in V} \left(\max_{u \in V} \delta(v, u) \right) = \text{rad}(G)$$

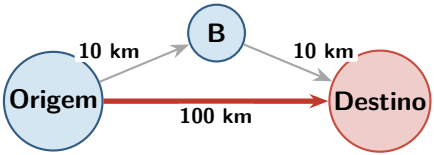
Garante equidade e tempo máximo tolerável em emergências!

Algoritmo em Big Data

1. Execute BFS a partir de cada nó $v \in V$ ($|V|$ vezes);
2. Compute o vetor de excentricidades em $\mathcal{O}(|V|(|V| + |E|))$;
3. Selecione o nó com menor $\epsilon(v)$.

O Choque Didático: Por Que o BFS Falha em Grafos Ponderados?

Em malhas rodoviárias reais, cada trecho possui sua quilometragem. Vamos testar o BFS em um grafo com pesos:



O que o BFS Decide?

- O BFS visita primeiro todos os nós a 1 salto;
- O destino C é vizinho direto de A!
- O BFS seleciona a rota direta: $\mathbf{A} \rightarrow \mathbf{C}$ com custo de **100 km**!

O Erro Grave

A rota via B possui 2 saltos, mas gasta apenas $\mathbf{10} + \mathbf{10} = \mathbf{20}$ km! O BFS errou por um fator de 5x!

Diagnóstico: A Miopia Estrutural da Fila FIFO

Por Que Isso Aconteceu?

A Fila FIFO assume tacitamente que:

$$\text{Menor Número de Saltos} \iff \text{Menor Distância Real}$$

Isso é verdade se e somente se todas as arestas tiverem peso idêntico ($w(e) = 1$). Quando há pesos arbitrários, uma aresta longa pode custar mais do que dezenas de arestas curtas!

O Que Precisamos Mudar?

Substituir a simples contagem de saltos por uma **estimativa acumulada de distância** e ordenar a exploração por uma **Fila de Prioridades (Min-Heap)**, extraindo sempre o vértice com menor custo acumulado!

A Operação Fundamental: Relaxamento de Aresta

Todos os algoritmos clássicos de caminho mínimo ponderado baseiam-se na técnica de **Relaxamento**:

Conceito de Estimativa de Limite Superior ($d[v]$)

Para cada vértice $v \in V$, mantemos $d[v]$ como o custo do menor caminho conhecido até o momento de s até v. Inicialmente:

$$d[s] = 0 \quad \text{e} \quad d[v] = \infty, \ \forall v \neq s$$

Definition (Relaxamento de (u, v) com peso $w(u, v)$)

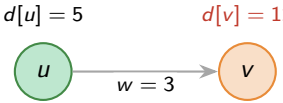
Testar se o caminho para v passando por u melhora o caminho atualmente conhecido:

$$\textbf{se } d[v] > d[u] + w(u, v) \textbf{ então:}$$

$$d[v] \leftarrow d[u] + w(u, v) \quad \text{e} \quad \pi[v] \leftarrow u$$

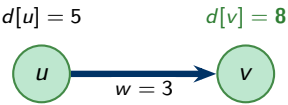
Ilustração do Relaxamento de Aresta

Antes do Relaxamento



Caminho via u: $5 + 3 = 8 < 12$.
Melhoria encontrada!

Depois do Relaxamento



$d[v]$ é atualizado para 8 e $\pi[v] = u$. Aresta relaxada com sucesso!

Edsger W. Dijkstra (1930–2002)

Origem Histórica (1956):

- Concebido em 1956 pelo pioneiro holandês da computação Edsger Dijkstra em apenas 20 minutos durante uma ida às compras com sua noiva;
- Publicado em 1959 no artigo clássico: *"A Note on Two Problems in Connexion with Graphs"*;
- Publicado originalmente sem computadores, para demonstrar o potencial de cálculo de rotas entre cidades holandesas.

Condição Crítica de Uso

Todos os pesos de aresta devem ser NÃO-NEGATIVOS:

$$w(u, v) \geq 0, \quad \forall (u, v) \in E$$

Pesos negativos quebram a premissa gulosa!

Estratégia Gulosa (Greedy Choice Property)

O Algoritmo de Dijkstra mantém a partição dos vértices em dois conjuntos:

- S: Conjunto de vértices cujas distâncias finais já estão **definitivamente fechadas**;
- $Q = V \setminus S$: Fila de prioridades com vértices cujas distâncias ainda são provisórias.

A Escolha Gulosa a Cada Passo

Extraia de Q o vértice u com a menor distância estimada $d[u]$:

$$u = \arg\min_{v \in Q} d[v]$$

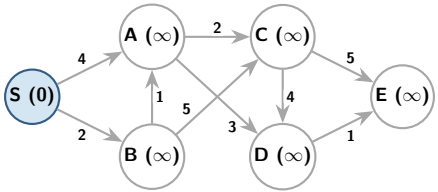
Garantia Teórica: Como não há pesos negativos, nenhuma rota futura poderá diminuir a distância até u passando por outros nós. Logo, $d[u] = \delta(s, u)$ é DEFINITIVO!

Pseudocódigo do Algoritmo de Dijkstra

Dijkstra(Grafo G, Origem s)

1. **Para cada** $v \in V$: $d[v] \leftarrow \infty$, $\pi[v] \leftarrow$ None; $\quad d[s] \leftarrow 0$
2. $Q \leftarrow$ Min-Heap com pares $(d[v], v)$; inserir $(0, s)$
3. **Enquanto** $Q \neq \emptyset$:
   - $(dist_u, u) \leftarrow$ Extrair-Mínimo(Q)
   - **Se** $dist_u > d[u]$: **continue** *(descarta duplicata obsoleta)*
   - **Para cada** vizinho v de u com aresta (u, v) de peso $w(u, v)$:
     - nova_dist $\leftarrow d[u] + w(u, v)$
     - **Se** nova_dist $< d[v]$:
       $d[v] \leftarrow$ nova_dist; $\quad \pi[v] \leftarrow u$; $\quad$ Inserir($Q, (d[v], v)$)

Dijkstra Passo a Passo: Grafo Didático & Passo 0



Min-Heap Q (topo $\rightarrow$): $(0, S)$

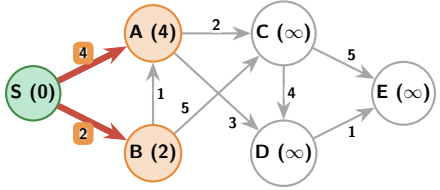
Passo 0: Inicialização das Estruturas

- Distâncias: $d[S] \leftarrow 0$; $d[v] \leftarrow \infty$ para todos os demais $v \in V$.
- Predecessores: $\pi[v] \leftarrow$ None para todo v.
- Inserir a origem $(0, S)$ no Min-Heap Q.
- Nenhum vértice foi fechado (consolidado) ainda.

Tabela Parcial de Distâncias

Vértice	S	A	B	C	D	E
$d[v]$	**0**	∞	∞	∞	∞	∞
$\pi[v]$	–	–	–	–	–	–

Dijkstra Passo a Passo: Passo 1 – Processando S



Min-Heap Q (topo $\rightarrow$): $(2, B)$ $(4, A)$

Passo 1: Extração e Relaxamento de S

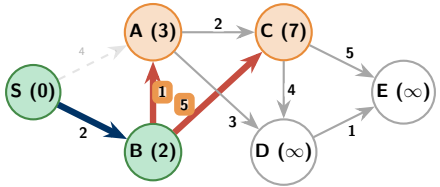
- Extrai o mínimo: $(0, S) \implies$ **S fechado** ($d[S] = 0$).
- Relaxa (S, A): $0 + 4 = \mathbf{4} < \infty \implies d[A] \leftarrow 4,\ \pi[A] \leftarrow S$.
- Relaxa (S, B): $0 + 2 = \mathbf{2} < \infty \implies d[B] \leftarrow 2,\ \pi[B] \leftarrow S$.
- Insere $(2, B)$ e $(4, A)$ no Min-Heap Q.

Tabela Parcial de Distâncias

Vértice	S	A	B	C	D	E
$d[v]$	**0**	**4**	**2**	∞	∞	∞
$\pi[v]$	–	S	S	–	–	–

Próxima decisão: Extrair o menor do heap $\implies$ B com distância 2.

Dijkstra Passo a Passo: Passo 2 – Processando B (Relaxa A!)



Min-Heap Q (topo $\rightarrow$):

$(3, A)$ $(4, A)^*$ $(7, C)$

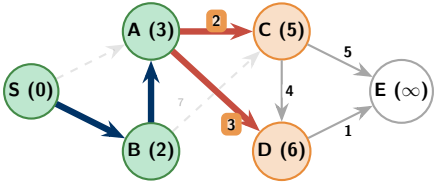
Passo 2: Relaxamento Crítico de A

- Extrai $(2, B) \implies$ **B fechado** ($d[B] = 2$). Aresta (S, B) confirmada na árvore!
- **Relaxa** (B, A)**:** $d[B] + w(B, A) = 2 + 1 = \mathbf{3} < \mathbf{4}$! $\implies d[A] \leftarrow 3,\ \pi[A] \leftarrow B$!
  (O atalho via B superou a aresta direta (S, A)!)
- **Relaxa** (B, C)**:** $2 + 5 = \mathbf{7} < \infty \implies d[C] \leftarrow 7,\ \pi[C] \leftarrow B$.
- Insere $(3, A)$ e $(7, C)$ no Min-Heap Q.

Tabela Parcial de Distâncias

Vértice	S	A	B	C	D	E
$d[v]$	**0**	**3**	**2**	**7**	∞	∞
$\pi[v]$	–	**B**	S	B	–	–

Dijkstra Passo a Passo: Passo 3 – Processando A (Relaxa C!)



Min-Heap Q (topo $\rightarrow$):

$(5, C)$ $(6, D)$ $(7, C)^*$

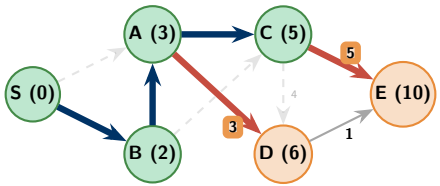
Passo 3: Extração de A e Relaxamento de C

- Extrai $(3, A) \implies$ **A fechado** ($d[A] = 3$). Aresta (B, A) confirmada!
- **Relaxa** (A, C): $d[A] + w(A, C) = 3 + 2 = \mathbf{5} < \mathbf{7}$! $\implies d[C] \leftarrow 5,\ \pi[C] \leftarrow A$!
  (A rota via A superou a rota anterior via B!)
- **Relaxa** (A, D):
  $3 + 3 = \mathbf{6} < \infty \implies d[D] \leftarrow 6,\ \pi[D] \leftarrow A.$
- Entrada duplicada $(4, A)^*$ descartada do heap.

Tabela Parcial de Distâncias

Vértice	S	A	B	C	D	E
$d[v]$	**0**	**3**	**2**	**5**	**6**	∞
$\pi[v]$	–	B	S	**A**	A	–

Dijkstra Passo a Passo: Passo 4 – Processando C (Descobre E)



Min-Heap Q (topo $\rightarrow$):

$(6, D)$ $(7, C)^*$ $(10, E)$

Passo 4: Extração de C e Descoberta de E

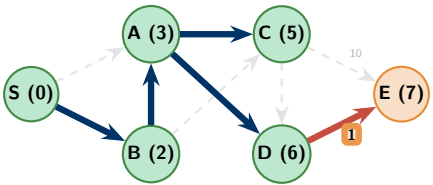
- Extrai $(5, C) \implies$ **C fechado** ($d[C] = 5$). Aresta (A, C) entra na árvore!
- (C, D): $5 + 4 = 9 \geq 6 \implies$ **Não relaxa** (rota via A é melhor).
- (C, E):
  $5 + 5 = \mathbf{10} < \infty \implies d[E] \leftarrow 10,\ \pi[E] \leftarrow C$.
- Insere $(10, E)$ no Min-Heap Q.

Tabela Parcial de Distâncias

Vértice	S	A	B	C	D	E
$d[v]$	**0**	**3**	**2**	**5**	**6**	**10**
$\pi[v]$	–	B	S	A	A	C

Próxima decisão: Extrair o menor do heap $\implies$ D com distância 6.

Dijkstra Passo a Passo: Passo 5 – Processando D (Relaxa E!)



Min-Heap Q (topo $\rightarrow$):

$(7, E)$ | $(7, C)^*$ | $(10, E)^*$

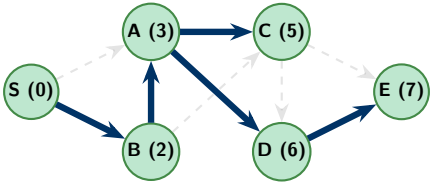
Passo 5: Relaxamento do Destino E

- Extrai $(6, D) \implies$ **D fechado** ($d[D] = 6$). Aresta (A, D) entra na árvore!
- **Relaxa** (D, E)**:**
  $d[D] + w(D, E) = 6 + 1 = \mathbf{7} < \mathbf{10}$!
  $\implies d[E] \leftarrow 7,\ \pi[E] \leftarrow D$!
  (A rota via D reduziu o custo até E de 10 para 7!)
- Insere $(7, E)$ no Min-Heap Q.

Tabela Parcial de Distâncias

Vértice	S	A	B	C	D	E
$d[v]$	**0**	**3**	**2**	**5**	**6**	**7**
$\pi[v]$	–	B	S	A	A	**D**

Dijkstra Passo a Passo: Passo 6 – Conclusão e Árvore Final



Min-Heap Q: *Vazio! (Entradas obsoletas descartadas)*

Árvore de Caminhos Mínimos Concluída!

- Extrai $(7, E) \implies$ **E fechado** ($d[E] = 7$). Aresta (D, E) confirmada!
- Entradas $(7, C)^*$ e $(10, E)^*$ são descartadas pois suas chaves excedem $d[C]$ e $d[E]$.
- Heap esvazia $\implies$ **Algoritmo finalizado com sucesso!**

Rotas Ótimas Finais a Partir de S

- $S \to B$: $\langle S, B \rangle$ (Custo: 2)
- $S \to A$: $\langle S, B, A \rangle$ (Custo: 3)
- $S \to C$: $\langle S, B, A, C \rangle$ (Custo: 5)
- $S \to D$: $\langle S, B, A, D \rangle$ (Custo: 6)
- $S \to E$: $\langle S, B, A, D, E \rangle$ (Custo: 7)

Rastreio Consolidado: Tabela de Distâncias de Dijkstra

Iteração	**Extrai** u	$d[S]$	$d[A]$	$d[B]$	$d[C]$	$d[D]$	$d[E]$
0 (Inicial)	–	**0**	∞	∞	∞	∞	∞
1	**S** (0)	**0**	4	**2**	∞	∞	∞
2	**B** (2)	**0**	**3**	**2**	7	∞	∞
3	**A** (3)	**0**	**3**	**2**	**5**	**6**	∞
4	**C** (5)	**0**	**3**	**2**	**5**	**6**	10
5	**D** (6)	**0**	**3**	**2**	**5**	**6**	**7**
6	**E** (7)	**0**	**3**	**2**	**5**	**6**	**7**

Observe os Dois Momentos Críticos de Relaxamento

- **Vértice A:** Na iteração 1, $d[A] = 4$ via (S, A). Na iteração 2, ao extrair B, o caminho $S \to B \to A$ custa $2 + 1 = 3 < 4$. $d[A]$ caiu para 3!
- **Vértice E:** Na iteração 4, $d[E] = 10$ via C. Na iteração 5, ao extrair D, o caminho via D custa $6 + 1 = 7 < 10$. $d[E]$ caiu para 7!

Complexidade Assintótica de Dijkstra

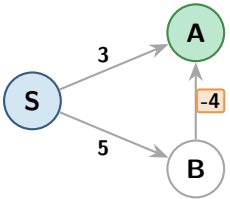
A complexidade depende diretamente da estrutura de dados usada para a Fila de Prioridades:

Fila de Prioridades	**Extrair-Mínimo**	**Diminuir-Chave**	**Complexidade Total**
Array / Vetor Simples	$\mathcal{O}(V)$	$\mathcal{O}(1)$	$\mathcal{O}(V^2)$
Min-Heap Binária (Python)	$\mathcal{O}(\log \mathbf{V})$	$\mathcal{O}(\log \mathbf{V})$	$\mathcal{O}((\mathbf{V} + \mathbf{E}) \log \mathbf{V})$
Fibonacci Heap (Teórica)	$\mathcal{O}(\log V)$	$\mathcal{O}(1)$ amortizado	$\mathcal{O}(E + V \log V)$

Regra de Engenharia de Software

Para grafos esparsos comuns no Agro ($|E| \ll |V|^2$), a implementação com `heapq` (Min-Heap Binária) é disparada a mais eficiente e padrão na indústria!

Por Que Pesos Negativos Quebram Dijkstra?



Onde o Algoritmo Falha?

1. Inicializa $d[S] = 0, d[A] = \infty, d[B] = \infty$.
2. Extrai S. Relaxa vizinhos: $d[A] = 3, d[B] = 5$.
3. O algoritmo é guloso: extrai A (menor: 3) e o **marca como fechado**!
4. Depois extrai B (5). A aresta (B, A) com peso -4 geraria: $5 + (-4) = 1 < 3$!
5. Como A já foi fechado, Dijkstra nunca reconsidera A!

Para arestas com pesos negativos, use Bellman-Ford ($\mathcal{O}(VE)$).

Métricas Topológicas via BFS em Python

```python
def metricas_topologicas(self) -> dict[str, Any]:
    """Calcula Excentricidade, Raio, Diametro e Centro do Grafo."""
    excentricidades = {}
    for v in self.vertices:
        dist, _ = self.bfs_caminho_minimo(v)
        excentricidades[v] = max(int(d) for d in dist.values() if d != float("inf"))

    raio = min(excentricidades.values())
    diametro = max(excentricidades.values())
    centro = [v for v, e in excentricidades.items() if e == raio]

    return {
        "excentricidades": excentricidades,
        "raio": raio,
        "diametro": diametro,
        "centro": centro,
    }

```

Algoritmo de Dijkstra com heapq em Python (1/2)

```python
import heapq

def dijkstra(self, origem: str) -> tuple[dict[str, float], dict[str, str | None], list[str]]:
    """Calcula caminhos minimos usando Min-Heap binaria (O((V+E) log V))."""
    dist = {v: float("inf") for v in self.vertices}
    pred = {v: None for v in self.vertices}
    visitados = set()
    ordem_fechamento = []

    dist[origem] = 0.0
    heap = [(0.0, origem)]  # Tuplas (distancia_acumulada, vertice)

    while heap:
        dist_u, u = heapq.heappop(heap)
        if u in visitados:
            continue  # Descarta entradas obsoletas do heap

        visitados.add(u)
        ordem_fechamento.append(u)
        # Continua no proximo slide...

```

Algoritmo de Dijkstra com heapq em Python (2/2)

```
        # ...continuacao do laco while:
        for v, peso_uv in self._adj[u].items():
            if v not in visitados:
                nova_dist = dist_u + peso_uv
                if nova_dist < dist[v]:
                    dist[v] = nova_dist
                    pred[v] = u
                    heapq.heappush(heap, (nova_dist, v))

    return dist, pred, ordem_fechamento

```

Garantia de Desempenho: A biblioteca padrão `heapq` do Python implementa uma Min-Heap binária em C, garantindo push e pop em tempo logarítmico estrito $\mathcal{O}(\log V)$.

Reconstrução da Rota Ótima em Python

```python
def caminho_minimo_dijkstra(self, origem: str, destino: str) -> tuple[list[str], float]:
    """Reconstroi o caminho de custo minimo entre origem e destino."""
    dist, pred, _ = self.dijkstra(origem)
    if dist[destino] == float("inf"):
        return [], float("inf")  # Destino inalcancavel

    rota = []
    atual = destino
    while atual is not None:
        rota.append(atual)
        atual = pred[atual]
    rota.reverse()
    return rota, dist[destino]

```

Caso 1: Roteamento Rodoviário Regional de Safra

Cenário Regional (Interior de SP):

- Transporte de colheita de citros de **Monte Azul Paulista** até o terminal rodoferroviário de exportação em **Ribeirão Preto**;
- Malha conectando Bebedouro, Barretos, Colina, Severínia, Olímpia, Pitangueiras e Sertãozinho;
- Rodovias com distâncias reais em quilômetros.

Resultado do Algoritmo

Rota Ótima Calculada:
Monte Azul → Bebedouro (32 km)
→ Pitangueiras (38 km)
→ Sertãozinho (36 km)
→ Ribeirão Preto (21 km)

Distância Total: 127.0 km

Impacto Financeiro

O algoritmo descartou a rota via Barretos (167 km) e via Jaboticabal (149 km), poupando R$ 180,00 por frete de carreta bitrem.

Caso 2: Localização Estratégica de Silo Regional

Problema de Decisão no Agronegócio:

- Instalação de um novo Silo de Transbordo de Fertilizantes para atender 8 municípios associados;
- Critério de decisão: **Minimizar o tempo máximo** que qualquer produtor leva para retirar insumos.

Cálculo de Centralidade

ϵ(Bebedouro) = 3 saltos (Mínimo!)
ϵ(Pitangueiras) = 3 saltos
ϵ(Monte Azul) = 4 saltos
ϵ(Barretos) = 5 saltos

Parecer Técnico do Engenheiro

- Raio rad(G) = **3** saltos
- Diâmetro diam(G) = **5** saltos
- **Centro** :{Bebedouro, Pitangueiras}

Instalar o Hub em **Bebedouro** assegura atendimento a toda a malha regional em no máximo 3 saltos logísticos!

Caso 3: Roteamento com Custo Composto (Diesel + Pedágio)

O Dilema do Frete Real:

- Carreta bitrem: Custo operacional de **R$ 4,50 / km** rodado (combustível + pneus + desgaste);
- Rodovias concessionadas cobram **tarifas de pedágio**;
- Frequentemente a rodovia mais curta em km possui tarifas elevadas, tornando-se mais cara no balanço final!

Comparativo de Rotas

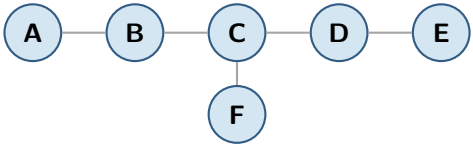
Rota 1 (Mais curta em km):
Via Ponto A: 90 km + R$ 80 pedágios
Custo Total = **R$ 485,00**

Rota 2 (Mais longa em km):
Via Ponto B: 95 km + R$ 10 pedágio
Custo Total = **R$ 437,50**

$$\text{Peso da Aresta} = (\text{km} \times 4.50) + \text{Pedágio}$$

Conclusão: Dijkstra escolhe a Rota 2, economizando **R$ 47,50** por viagem!

Exercício 1: Excentricidade, Raio, Diâmetro e Centro



Desafio para a Turma

Para o grafo não-ponderado acima:

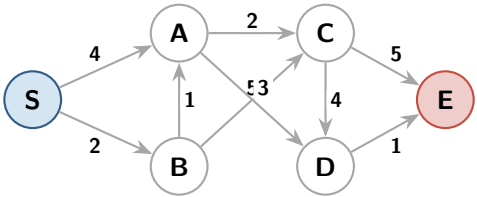
1. Determine a excentricidade $\epsilon(v)$ de cada um dos 6 vértices.
2. Identifique o Raio $\text{rad}(G)$.
3. Identifique o Diâmetro $\text{diam}(G)$.
4. Encontre o Centro $Z(G)$.

Exercício 1: Resolução Passo a Passo

Gabarito Comentado

1. **Excentricidades:**
   - A: distância máxima até E é **4** ($\langle A, B, C, D, E \rangle$) $\implies \epsilon(A) = 4$
   - B: distância máxima até E é **3** ($\langle B, C, D, E \rangle$) $\implies \epsilon(B) = 3$
   - C: distâncias máximas são A (2), E (2), F (1) $\implies \epsilon(\mathbf{C}) = \mathbf{2}$
   - D: distância máxima até A é **3** ($\langle D, C, B, A \rangle$) $\implies \epsilon(D) = 3$
   - E: distância máxima até A é **4** ($\langle E, D, C, B, A \rangle$) $\implies \epsilon(E) = 4$
   - F: distâncias até A e E são **3** $\implies \epsilon(F) = 3$
2. **Raio:** $\text{rad}(G) = \min\{4, 3, 2, 3, 4, 3\} = \mathbf{2}$.
3. **Diâmetro:** $\text{diam}(G) = \max\{4, 3, 2, 3, 4, 3\} = \mathbf{4}$.
4. **Centro:** Vértices com $\epsilon = 2 \implies \mathbf{Z(G)} = \{\mathbf{C}\}$.

Exercício 2: Execução Manual de Dijkstra



Desafio para a Turma

Execute o algoritmo de Dijkstra com origem em S:

1. Preencha a tabela de distâncias em cada iteração do Min-Heap;
2. Identifique a ordem de fechamento definitivo dos vértices;
3. Apresente o caminho mínimo e o custo total de S até E.

Exercício 2: Resolução Passo a Passo

Gabarito Oficial de Dijkstra a Partir de S

1. **Ordem de Fechamento Definitivo pelo Min-Heap:**
   $\mathbf{S} \xrightarrow{(0)} \mathbf{B} \xrightarrow{(2)} \mathbf{A} \xrightarrow{(3)} \mathbf{C} \xrightarrow{(5)} \mathbf{D} \xrightarrow{(6)} \mathbf{E} \xrightarrow{(7)}$
2. **Tabela Final de Distâncias e Predecessores:**
   - $S : d[S] = 0, \quad \pi[S] = \text{None}$
   - $B : d[B] = 2, \quad \pi[B] = S$
   - $A : d[A] = 3, \quad \pi[A] = B \quad$ *(via B, pois $2 + 1 < 4$)*
   - $C : d[C] = 5, \quad \pi[C] = A \quad$ *(via A, pois $3 + 2 < 7$)*
   - $D : d[D] = 6, \quad \pi[D] = A \quad$ *(via A, pois $3 + 3 = 6$)*
   - $E : d[E] = 7, \quad \pi[E] = D \quad$ *(via D, pois $6 + 1 < 10$)*
3. **Caminho Mínimo até E:**
   $\mathbf{S} \to \mathbf{B} \to \mathbf{A} \to \mathbf{D} \to \mathbf{E}$ com **Custo Total = 7.0**!

Resumo da Aula: O Que Aprendemos Hoje?

Caminho Mínimo Não-Ponderado

- BFS garante menor número de saltos em $\mathcal{O}(V + E)$;
- Métricas de centralidade baseadas em caminhos: Excentricidade, Raio e Diâmetro;
- O Centro do Grafo ($Z(G)$) é o local ótimo para Hubs e Silos.

Algoritmo de Dijkstra

- O BFS falha em grafos ponderados devido à miopia da fila FIFO;
- Dijkstra utiliza Relaxamento e Fila de Prioridades Min-Heap ($\mathcal{O}((V + E) \log V)$);
- Exige pesos estritamente não-negativos ($w \geq 0$).

"Calcular a menor distância não é apenas sobre geometria; é a arte matemática de poupar recursos, tempo e energia em escala global."

Próximos Passos na Disciplina (Cronograma SIGA)

Sequência Oficial das Próximas Aulas

- **Aula 29: Backtracking**
  - Estratégia de busca com retrocesso e espaço de estados;
  - Problema das N-Rainhas, labirintos e poda de ramos inviáveis.
- **Aula 30: Algoritmos Gulosos (Greedy Algorithms)**
  - Princípio guloso e prova de corretude;
  - Problema da Seleção de Atividades e Fracionamento.
- **Aula 31: Árvores Geradoras Mínimas (MST)**
  - Algoritmos de Prim e Kruskal para eletrificação e irrigação rural.

Atividade Prática de Laboratório

Executar `python aula14_caminho_minimo.py` e adicionar uma nova rodovia na malha regional para verificar se a rota de Monte Azul a Ribeirão Preto se altera.

Referências Bibliográficas

- CORMEN, Thomas H. et al. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2012. (Cap. 24: Caminhos Mínimos de Origem Única).
- DIJKSTRA, Edsger W. **A Note on Two Problems in Connexion with Graphs**. Numerische Mathematik, v. 1, p. 269–271, 1959.
- WEST, Douglas B. **Introduction to Graph Theory**. 2. ed. Upper Saddle River: Prentice Hall, 2001.
- SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. Addison-Wesley, 2011.
- DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos; VAZIRANI, Umesh. **Algoritmos**. McGraw-Hill, 2008.
- SZWARCFITER, Jayme Luiz. **Grafos e Algoritmos Computacionais**. Rio de Janeiro: Campus, 1984.

Bons Estudos!

Dúvidas ou Perguntas?

douglas.goncalves01@cps.sp.gov.br